

• netwatch

看什么程序在偷偷上传、传给谁、传了多少。

进程级归因

实时仪表盘

异常告警

只读元数据

进程级上传监控仪表盘 • 本机常驻采集 + Web 实时查看

它回答四个问题

01

现在谁在传？

实时视图：每个进程 → 每个远端地址的上传速率，SSE 秒级推送

02

它传给谁？

IP → 域名 → 归属地 / ASN / 服务分类，一眼判断“该不该传”

03

它传了多少？

分钟粒度聚合落库，1h / 24h / 7d 按进程、目的地双维度回看

04

什么在偷偷传？

新目的地首传、超阈值大流量、归因失败外传，三条规则自动告警

只观测，不干预

检测靠行为元数据 —— 谁 · 何时 · 何地 · 多少

做

- 进程级流量归因 流量归到「进程 × 目的地」两级
- 目的地富化 域名 / 归属地 / ASN / 服务分类
- 行为异常告警 新目的地 · 超阈值 · 归因失败
- 历史回看与查询 分钟粒度 90 天, CLI 与 Web 双入口

不做

- 内容审计与 MITM 解密
- 防火墙与拦截 (opensnitch 的领域)
- 跨主机 / 集群监控 (只看本机)

发现问题的手段是“看得见”，不是“管得住”。

实时视图 • 现在谁在传

进程卡

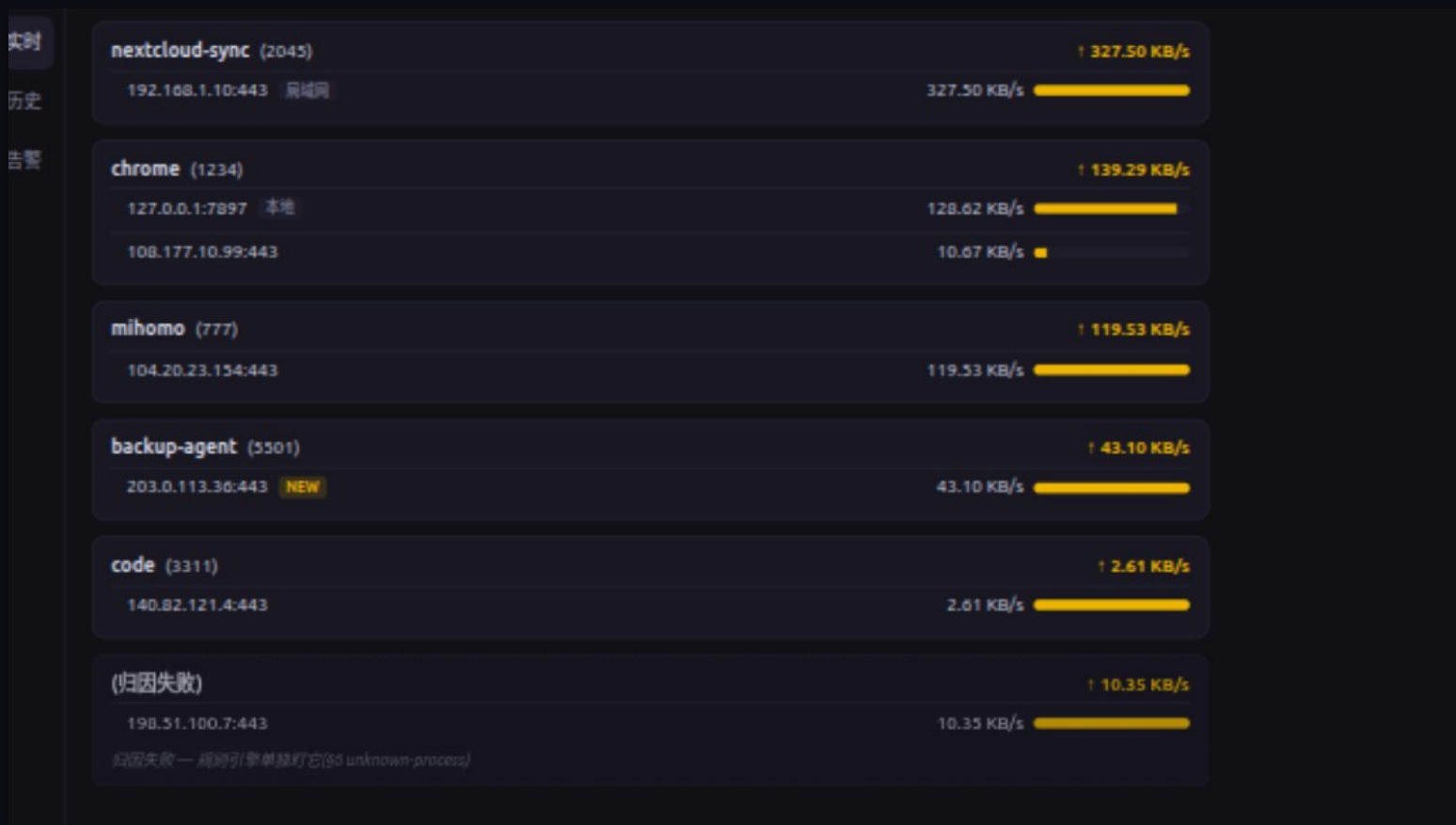
每个在传数据的进程一张卡，展开是目的地明细

徽标

局域网 / 本地 (代理) / NEW 首见 / 归因失败，
状态一眼可读

顶栏

全局上下行速率 + 未读告警，SSE 秒级推送，
断线自动重连



mock 数据源 • 真实界面 (环回代理两跳形态 :chrome → 127.0.0.1:7897 → mihome → 公网)

两个进程，一个库

采集进程 需 root

ss 轮询 → 行解析 → 事件流

富化器 (域名 / 代理两跳)

聚合器 (分钟桶) → 落库

规则引擎 → 桌面 / webhook

SQLite
WAL · 唯一耦合点

展示进程 普通用户

Hono API (REST + SSE)

React 仪表盘 · 三视图

只读打开数据库

规则配置 · 白名单管理

采集端不开任何网络端口 · 明细保留 90 天自动清理 · 采集崩了不影响看历史, 展示重启不丢数据

采集：为什么是 ss 轮询

Phase 0 实证

nethogs -t 输出的是每进程聚合，没有远端地址列 —— 撑不起“传给谁”，原方案被真实样本推翻

ss 的答案

每条连接自带远端地址、进程归属和累计字节；相邻两拍做差即增量，零抓包、零 pcap 依赖

nethogs 没有缺席

降级为交叉验证 / 无目的地降级源，与 ss 同一 TrafficSource 契约

已知局限：两次轮询之间建立又关闭的短连接会漏；UDP 无字节计数 —— eBPF 精确统计为后续进阶路线

```
$ ss -tinp state established
Peer Address:Port    104.20.23.154:443
users:(("mihomo",pid=777,fd=48))
bytes_sent:2097152  bytes_received:838860
```

真实输出字段 (节选): 远端地址 + 进程归属 + 每连接累计字节

富化：把 IP 翻译成人话

反向 DNS 已实现

108.177.10.99 → oy-in-f99.1e100.net(本机发起 , 正 / 负缓存 6h)

分类规则表 已实现

静态后缀匹配 :AI 服务 / 网盘 / 对象存储徽标 —— “谁在往 AI 服务传文件” 一眼可见

GeoLite2 归属地 / ASN 接入中

只用本地库文件 ,netwatch 自己不做在线查询

环回代理两跳：告警归因不被代理污染



应用的 remotep 是代理而非真实远端；富化器按字节相关启发式 (±25% / ±5s) 把两跳关联 ,transit 代理进程豁免新目的地告警。

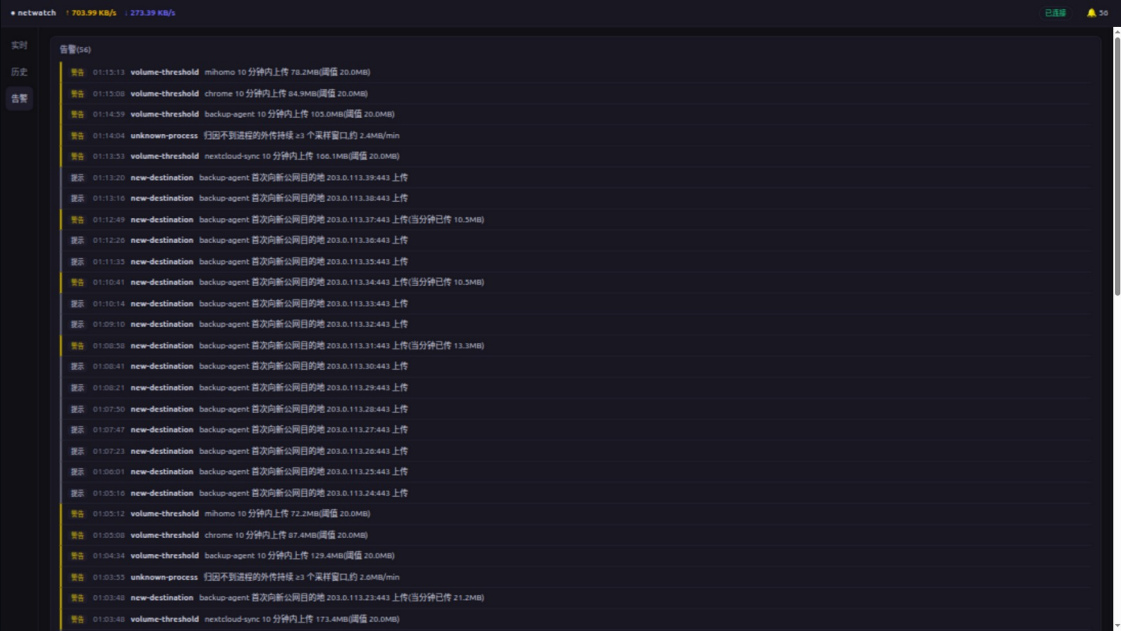
关联只影响展示与告警标注，不改变底层归因事实。

三条规则，盯住“偷偷传”

规则	触发条件（默认，可配）
new-destination	已知进程首次向新公网 IP 上传；当分钟 > 10MB 升 warn
volume-threshold	单进程 100MB/10min 或 500MB/h
unknown-process	归因不到进程的持续外传：连续 3 窗口且 > 1MB/min

白名单 进程 × 目的 IP 命中即静默 (支持 * 进程级); 已确认的代理中转进程豁免 new-destination

通知 桌面 notify-send + webhook, 告警入库供仪表盘展示；通知失败静默，不影响采集



演示运行 35 分钟 (mock 数据源):56 条告警，三条规则全部命中

数据与隐私

所有数据不出这台机器

- **只记录元数据** IP、端口、域名、字节数、时间 —— 不接触任何传输内容, 不做 MITM
- **GeoIP 全部本地库** 本工具监控别人联网, 自己不能偷偷联网: 归属地 / ASN 走本地文件, 域名走本机 DNS 反向解析
- **单个 SQLite 文件** 明细保留 90 天, 采集端每小时自动清理; 规则配置与数据库都是本机普通文件

数据库 `~/.local/share/netwatch/netwatch.db` 规则 `~/.config/netwatch/rules.json`

平台支持与当前状态

Linux

首发

ss 轮询主路径 + nethogs 交叉验证 ,eBPF 为后续进阶

macOS

Phase 7

nettop -P -L(系统自带 , 零依赖)

Windows

Phase 8

ETW; 降级为连接轮询 (无字节数)

平台差异全部封在采集适配层 (单一 TrafficSource 契约), 上层零感知 ; 降级由 capabilities 声明驱动。

当前状态

- ✓ Phase 0~4: 采集 → 落库 → 查询 → 规则 → 仪表盘 ,mock 全链路验证
- ✓ 真实采集源 (ss 轮询) 端到端验证 ;nethogs 解析器按 0.8.7 校准
- Phase 5 进行中 : 两跳富化 • 反向 DNS • 90 天清理 • systemd unit
- 待办 :GeoLite2 接入 • DNS 观察 • root 实机验收 (30 分钟稳定性)

快速体验

```
npm install
npm run collector:mock          # 无 root 体验全链路(mock 数据源)
npm run web:build && npm run web # 仪表盘 http://127.0.0.1:8787
npm run query -- top 24h --by=destination
sudo npm run collector          # 真实采集(Linux 主路径:ss 轮询, 需 root)
```